

Definition: IR Decoder for Universal Graph-Token DSL

IR Decoder for Universal Graph-Token DSL

This page defines the **canonical IR decoder** that reconstructs Lean DSL graph structures from graph-token strings. It is the inverse of the IR encoder and completes the bijective mapping required for reversible $\text{Lean} \rightarrow \text{IR} \rightarrow \text{Lean}$ transformation.

This is **Layer 2B** of the architecture.

1. Utility Functions

Split on colon

```
def splitOnColon (s : String) : List String :=  
  s.splitOn ":"
```

Parse polarity

```
def parsePolarity (s : String) : Option Polarity :=  
  match s with  
  | "+" => some Polarity.pos  
  | "0" => some Polarity.zero  
  | "-" => some Polarity.neg  
  | _ => none
```

2. Parse Entity Identifiers

```
def parseEntityId (s : String) : EntityId :=
  if s.startsWith "N" then
    EntityId.num (s.drop 1 |>.toInt!)
  else if s.startsWith "T" then
    EntityId.term (s.drop 1 |>.toNat!)
  else if s = "EQ" then
    EntityId.eqn
  else if s.startsWith "F" then
    EntityId.fn (s.drop 1 |>.toNat!)
  else if s.startsWith "M" then
    EntityId.mat (s.drop 1 |>.toNat!)
  else if s.startsWith "V" then
    EntityId.vec (s.drop 1 |>.toNat!)
  else if s.startsWith "S" then
    EntityId.set (s.drop 1 |>.toNat!)
  else if s.startsWith "RV" then
    EntityId.rv (s.drop 2 |>.toNat!)
  else if s.startsWith "Op" then
    EntityId.op (s.drop 2 |>.toNat!)
  else if s.startsWith "Obj" then
    EntityId.obj (s.drop 3 |>.toNat!)
  else if s.startsWith "Mor" then
    EntityId.mor (s.drop 3 |>.toNat!)
  else
    EntityId.var s
```

3. Parse Relation Operators

```
def parseRelationOp (s : String) : RelationOp :=
  match s with
  | "mul" => RelationOp.mul
  | "add" => RelationOp.add
  | "sub" => RelationOp.sub
  | "div" => RelationOp.div
  | "eq" => RelationOp.eq
  | "diff" => RelationOp.diff
  | "integrate" => RelationOp.integrate
  | "limit" => RelationOp.limit
  | "eval_at" => RelationOp.eval_at
  | "matmul" => RelationOp.matmul
  | "dot" => RelationOp.dot
  | "cross" => RelationOp.cross
  | "transpose" => RelationOp.transpose
  | "inv" => RelationOp.inv
  | "det" => RelationOp.det
  | "union" => RelationOp.union
  | "intersect" => RelationOp.intersect
  | "set_sub" => RelationOp.set_sub
  | "subset" => RelationOp.subset
  | "and" => RelationOp.and
  | "or" => RelationOp.or
  | "not" => RelationOp.not
  | "implies" => RelationOp.implies
  | "iff" => RelationOp.iff
  | "prob" => RelationOp.prob
  | "expect" => RelationOp.expect
  | "var" => RelationOp.var
  | "dist" => RelationOp.dist
  | "compose" => RelationOp.compose
  | "id" => RelationOp.id
  | "assoc" => RelationOp.assoc
  | _ => RelationOp.custom s
```

4. Parse Operation Operators

```
def parseOperationOp (s : String) : OperationOp :=
  match s with
  | "add" => OperationOp.add
  | "sub" => OperationOp.sub
  | "div" => OperationOp.div
  | "iso" => OperationOp.iso
  | "apply_diff_rule" => OperationOp.apply_diff_rule
  | "apply_int_rule" => OperationOp.apply_int_rule
  | "change_var" => OperationOp.change_var
  | "limit_simplify" => OperationOp.limit_simplify
  | "row_reduce" => OperationOp.row_reduce
  | "diag" => OperationOp.diag
  | "eig_decomp" => OperationOp.eig_decomp
  | "simplify_logic" => OperationOp.simplify_logic
  | "simplify_set" => OperationOp.simplify_set
  | "normalize_formula" => OperationOp.normalize_formula
  | _ => OperationOp.custom s
```

5. Parse Entity Tokens

```
def parseEntityToken (s : String) : Option Entity :=
  match s.toList with
  | 'E' :: p :: ':' :: rest =>
    let pol? := parsePolarity (String.singleton p)
    match pol? with
    | none => none
    | some pol =>
      let body := String.mk rest
      let idStr := body.dropRight 1
      let chain := "*"
      some { class := "E", polarity := pol, id := parseEntityId idStr, chain := chain }
  | _ => none
```

6. Parse Attribute Tokens

```
def parseAttributeToken (s : String) : Option Attribute :=
  match s.toList with
  | 'A' :: p :: ':' :: rest =>
    let pol? := parsePolarity (String.singleton p)
    match pol? with
    | none => none
    | some pol =>
      let body := String.mk rest
      let parts := splitOnColon body
      match parts with
      | idChain :: key :: value :: _ =>
        let idStr := idChain.dropRight 1
        let chain := "*"
        some {
          class := "A",
          polarity := pol,
          target := parseEntityId idStr,
          key := key,
          value := value,
          chain := chain
        }
      | _ => none
  | _ => none
```

7. Parse Entity References

```
def parseEntityRef (s : String) : Option Entity :=
  parseEntityToken s
```

8. Parse Relation Tokens

```
def parseRelationToken (s : String) : Option Relation :=
  match s.toList with
  | 'R' :: p :: ':' :: rest =>
    let pol? := parsePolarity (String.singleton p)
    match pol? with
    | none => none
    | some pol =>
      let body := String.mk rest
      let parts := splitOnColon body
      match parts with
      | opChain :: e1Str :: e2Str :: e3Str :: _ =>
        let opName := opChain.dropRight 1
        let chain := "*"
        match parseEntityRef e1Str, parseEntityRef e2Str, parseEntityRef e3Str with
        | some e1, some e2, some e3 =>
          some {
            class := "R",
            polarity := pol,
            op := parseRelationOp opName,
            e1 := e1,
            e2 := e2,
            e3 := e3,
            chain := chain
          }
        | _, _, _ => none
      | _ => none
    | _ => none
```

9. Parse Operation Tokens

```
def parseOperationToken (s : String) : Option Operation :=
  match s.toList with
  | 'O' :: p :: ':' :: rest =>
    let pol? := parsePolarity (String.singleton p)
    match pol? with
    | none => none
    | some pol =>
      let body := String.mk rest
      let parts := splitOnColon body
      match parts with
      | opChain :: e1Str :: e2Str :: _ =>
        let opName := opChain.dropRight 1
        let chain := "*"
        match parseEntityRef e1Str, parseEntityRef e2Str with
        | some e1, some e2 =>
          some {
            class := "O",
            polarity := pol,
            op := parseOperationOp opName,
            e1 := e1,
            e2 := e2,
            chain := chain
          }
        | _, _ => none
      | _ => none
  | _ => none
```

10. Full Graph Decoder

```
def decodeGraph (tokens : List String) : Graph :=
  let step (acc : Graph) (t : String) : Graph :=
    match t.toList.head? with
    | some 'E' =>
      match parseEntityToken t with
      | some e => { acc with entities := e :: acc.entities }
      | none => acc
    | some 'A' =>
      match parseAttributeToken t with
      | some a => { acc with attributes := a :: acc.attributes }
      | none => acc
    | some 'R' =>
      match parseRelationToken t with
      | some r => { acc with relations := r :: acc.relations }
      | none => acc
    | some 'O' =>
      match parseOperationToken t with
      | some o => { acc with operations := o :: acc.operations }
      | none => acc
    | _ => acc
  tokens.foldl step { entities := [], attributes := [], relations := [], operations := [] }
```

Purpose of This Page

This decoder completes the reversible mapping between Lean DSL and the universal graph-token IR. It enables:

- round-trip Lean $\rightarrow$ IR $\rightarrow$ Lean
- SLM output reintegration
- verified rewrite pipelines
- VS Code autocompletion

This page is the authoritative reference for the IR decoding layer.